Ops 4: Container Orchestration

Minecraft on Kubernetes (k3s) - CS 312 System Administration

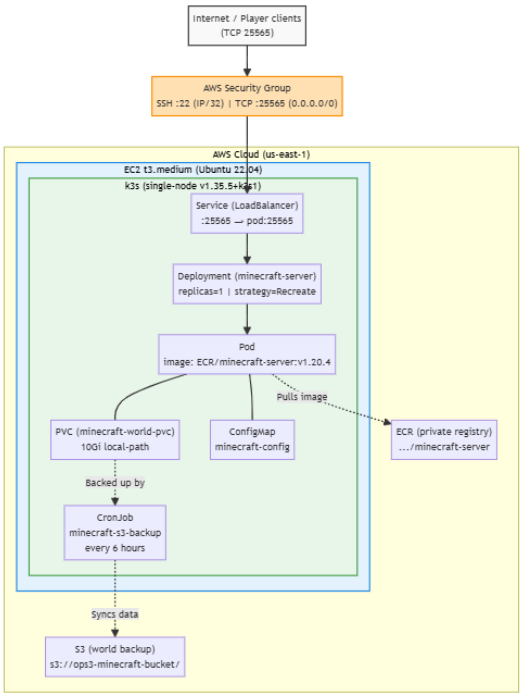
Dean Leon · Student ID: 934513949

Repository	https://github.com/CS-312-001-S2026/minecraft-ops-GenericProgram
Video	https://media.oregonstate.edu/media/t/1_hoshsdsl
Checkpoints	CP1: 00:04 · CP2: 00:28 · CP3: 01:07 · CP4: 01:57
AWS Region	us-east-1
Instance	t3.medium · Ubuntu 22.04 LTS · 20 GB gp3
Kubernetes	k3s v1.35.5+k3s1 (single-node)
Image	998487032255.dkr.ecr.us-east-1.amazonaws.com/minecraft-server:v1.20.4

This document was made with Python's ReportLab library. LLMs were utlised to format the document, ensure that all required sections were included, and to verify the correctness of content. The architecture diagram was created with Mermaid.js and exported as a PNG. The video walkthrough was edited to remove sections of waiting in order to meet the 3 minute requirement.

1. Architecture

The diagram below shows all components. Terraform provisions the EC2 host and security group. Ansible installs k3s, configures the ECR credential provider, restores world data from S3, and applies Kubernetes manifests. All AWS API calls from the node use the attached **LabInstanceProfile** IAM role, no credentials are hardcoded anywhere.



2. Repository File Map

All submission files are in the **ops3-4/** directory of the repository.

ops3-4/main.tf	Terraform: EC2 instance, security group, Ansible trigger
ops3-4/variables.tf	Terraform: input variable declarations
ops3-4/terraform.tfvars	Terraform: key name, PEM path, SSH CIDR
ops3-4/playbook.yml	Ansible: k3s install, ECR credential provider, S3 restore, manifest apply
ops3-4/k8s/configmap.yml	Kubernetes ConfigMap, EULA, VERSION, MOTD, MEMORY
ops3-4/k8s/pvc.yml	Kubernetes PersistentVolumeClaim, 10Gi local-path
ops3-4/k8s/deployment.yml	Kubernetes Deployment, probes, resources, volume mount
ops3-4/k8s/service.yml	Kubernetes Service, LoadBalancer :25565
ops3-4/k8s/backup-cronjob.yml	Kubernetes CronJob, S3 backup every 6 hours
ops3-4/checklist.sh	Video checkpoint command reference

3. Deployment Runbook

An operator with no prior knowledge of this setup should be able to deploy the server from scratch by following these steps in order.

3.1 Prerequisites

- AWS Academy lab started, CLI credentials pasted into **~/.aws/credentials**
- SSH key pair **cs312-key** exists in AWS and **~/.ssh/cs312-key.pem** exists locally
- Terraform ≥ 1.5, Ansible ≥ 2.14, Docker, nmap installed locally
- ECR repository **minecraft-server** exists, images **v1.20.4** and **v1.20.5** pushed
- S3 bucket **ops3-minecraft-bucket** exists (world data or empty)

3.2 Push images to ECR (first time or after session reset)

```
aws ecr create-repository --repository-name minecraft-server --region us-east-1
aws ecr get-login-password --region us-east-1 | \
  docker login --username AWS --password-stdin \
  998487032255.dkr.ecr.us-east-1.amazonaws.com

docker pull itzg/minecraft-server:java21
docker tag itzg/minecraft-server:java21 \
  998487032255.dkr.ecr.us-east-1.amazonaws.com/minecraft-server:v1.20.4
docker push 998487032255.dkr.ecr.us-east-1.amazonaws.com/minecraft-server:v1.20.4

# Tag same image as v1.20.5 for rollout demo
docker tag ...v1.20.4 ...v1.20.5 && docker push ...v1.20.5
```

3.3 Provision infrastructure

```
cd ops3-4/
# Set your IP in terraform.tfvars: ssh_allowed_cidr = "YOUR.IP/32"
terraform init
terraform apply
# Terraform provisions EC2, then auto-runs Ansible.
# Full run takes ~3-5 minutes. Note the output: server_public_ip
```

If Ansible fails partway through, re-run it directly: ANSIBLE_HOST_KEY_CHECKING=False ansible-playbook -i ',' -u ubuntu --private-key ~/.ssh/cs312-key.pem playbook.yml

3.4 Verify deployment

```
ssh -i ~/.ssh/cs312-key.pem ubuntu@
sudo kubectl get nodes          # STATUS = Ready
sudo kubectl get pods           # 1/1 Running (may take 90s)
sudo kubectl get svc            # minecraft-service EXTERNAL-IP visible

# From local machine:
nmap -sV -Pn -p T:25565
```

3.5 Service exposure on port 25565

The Kubernetes Service is type **LoadBalancer**. On single-node k3s, the built-in ServiceLB (klipper-lb) binds port 25565 directly on the host's network interface. The AWS Security Group allows TCP 25565 from 0.0.0.0/0. No Ingress controller or NodePort mapping is needed, players connect directly to the EC2 public IP on 25565.

3.6 Rollout to new image version

```
sudo kubectl set image deployment/minecraft-server \
  minecraft=998487032255.dkr.ecr.us-east-1.amazonaws.com/minecraft-server:v1.20.5

sudo kubectl rollout status deployment/minecraft-server
sudo kubectl rollout history deployment/minecraft-server
```

3.7 Rollback to previous version

```
sudo kubectl rollout undo deployment/minecraft-server
sudo kubectl rollout status deployment/minecraft-server

# Verify server is joinable after rollback:
nmap -sV -Pn -p T:25565
```

4. Backup and Restore

4.1 Backup procedure

World data is backed up to S3 automatically by a Kubernetes CronJob that runs every 6 hours. The job mounts the same PVC as the Minecraft pod (read-only) and runs **aws s3 sync** using credentials from the node's IAM instance profile.

To trigger a manual backup immediately:

```
sudo kubectl create job --from=cronjob/minecraft-s3-backup manual-backup-$(date +%s)
sudo kubectl get jobs
sudo kubectl logs job/manual-backup-
```

To verify backup contents in S3:

```
aws s3 ls s3://ops3-minecraft-bucket/ --recursive --human-readable
```

4.2 Step-by-step restore from S3

Follow these steps exactly to restore world data on a fresh instance. The Ansible playbook runs the S3 sync *before* applying Kubernetes manifests, so the PV is pre-populated when the pod first starts.

1. Start AWS Academy lab session and refresh ~/.aws/credentials on your local machine.
2. Confirm S3 bucket has data: **aws s3 ls s3://ops3-minecraft-bucket/**
3. Run **terraform apply** from ops3-4/. Ansible will automatically sync S3 → /opt/minecraft-data/ on the EC2 host.
4. Ansible then applies all Kubernetes manifests. The Deployment mounts the PVC at /data.
5. k3s local-path provisioner creates the PV directory under /var/lib/rancher/k3s/storage/ and the pod's /data is bind-mounted there.
6. SSH into EC2 and verify: **sudo kubectl exec <pod> -- ls /data**, you should see world/, server.properties, etc.
7. Connect with nmap or Minecraft client to confirm the server is running with restored world.

If restoring to an existing instance (no terraform apply): SSH in, run 'aws s3 sync s3://ops3-minecraft-bucket/ /opt/minecraft-data/', then 'sudo kubectl rollout restart deployment/minecraft-server'.

5. Tradeoff Notes

5.1 Workload controller: Deployment vs StatefulSet

A **Deployment** with **strategy: Recreate** is used rather than a StatefulSet. A StatefulSet provides stable network identity and ordered pod management, which are valuable for clustered databases. Minecraft is a single-replica stateful workload with no peer discovery requirement, the stable hostname benefit of StatefulSet does not apply. A Deployment with Recreate strategy is simpler to operate and sufficient: it guarantees the old pod is fully terminated before the new one starts, preventing two processes from simultaneously holding the ReadWriteOnce PVC.

5.2 Persistence: local-path PVC vs cloud-managed EBS

The k3s default **local-path** storage class provisions volumes on the node's local disk under /var/lib/rancher/k3s/storage/. This is simple and zero-cost. The tradeoff is that data is tied to the node: if the EC2 instance is *terminated* (not just rebooted), the PV data is lost. This is acceptable for this assignment because: (1) the assignment scope is single-node k3s,

(2) world data is backed up to S3 every 6 hours, and (3) the restore procedure is documented and tested. In a production environment, an EBS-backed StorageClass (e.g. the AWS EBS CSI driver) would be preferred for node-independent durability.

5.3 Service exposure: LoadBalancer vs NodePort

Service type **LoadBalancer** is used. On single-node k3s, ServiceLB (klipper-lb) binds the service port directly on the host without requiring an external load balancer. NodePort was avoided because it maps to a high port (30000-32767), requiring an extra firewall rule and forcing players to use a non-standard port. LoadBalancer keeps port 25565 standard and the security group rule minimal. Ingress is HTTP/HTTPS only and not applicable to the Minecraft TCP protocol.

5.4 Probe configuration

All three probe types are configured using **tcpSocket** on port 25565. A TCP socket check is a weaker health signal than a true application-level check (an open port does not guarantee the server has finished loading the world), but Minecraft exposes no HTTP health endpoint. The **startupProbe** allows up to 10 × 30s = 300 seconds for initial JVM and world loading before liveness/readiness checks begin, preventing restart loops during warmup. Once the startupProbe succeeds, the livenessProbe restarts the pod if the port stops responding (crash detection), and the readinessProbe gates Service traffic until the port is open.

5.5 Resource requests and limits

Requests (1 CPU, 2 Gi memory) guarantee the scheduler places the pod on a node with sufficient capacity and prevent eviction under normal load. Limits (2 CPU, 2500 Mi) cap resource consumption: the MEMORY env var sets the JVM heap to 2G, leaving ~500 Mi for JVM overhead and the OS. The CPU limit of 2 allows bursting during chunk generation without starving other system processes. On a t3.medium (2 vCPU, 4 GB), these limits leave ~1.5 GB for k3s system pods and the OS.

6. Teardown Checklist

Complete all steps below after finishing the assignment to prevent runaway AWS credit consumption.

■	Run **terraform destroy** from ops3-4/ to terminate the EC2 instance and delete the security group.
■	Confirm EC2 instance is terminated in the AWS Console (EC2 -> Instances).
■	Confirm the security group **minecraft_security_group_ops4** has been deleted.
■	Optionally delete the ECR repository to avoid storage costs: **aws ecr delete-repository --repository-name minecraft-server --force --region us-east-1**
■	Optionally empty and delete the S3 backup bucket if world data is no longer needed: **aws s3 rb s3://ops3-minecraft-bucket --force**
■	Click **End Lab** in the AWS Academy Learner Lab interface.
■	Verify credit balance the next day, AWS Academy credits update daily, not in real time.
■	Delete local terraform.tfstate if it contains sensitive output values.

Cost controls in place during the assignment:

- **Instance size:** t3.medium (~$0.0416/hr), minimum viable for Minecraft + k3s overhead
- **Stop schedule:** End Lab after every session, terraform destroy when not recording
- **SSH restriction:** Security group limits port 22 to a single known IP (/32 CIDR)
- **No RDS or NAT gateway:** default VPC used, no additional billable network services
- **ECR/S3:** minimal storage, well under free tier thresholds for short assignment duration